

Task Scheduler

Problem Statement: You are given a list of tasks represented by uppercase English letters ('A' to 'Z'), and an integer n representing a cooldown interval between two same tasks. Each task takes exactly 1 CPU interval to complete. Tasks can be executed in any order, but identical tasks must be separated by at least n intervals, during which the CPU may remain idle or execute other tasks. Return the minimum number of CPU intervals required to complete all the tasks .

Examples

Input : tasks = ["A","A","A","B","B","B"], n = 2

Output : 8

Explanation : One valid execution order is:

A -> B -> idle -> A -> B -> idle -> A -> B

Total intervals = 8

Input : tasks = ["A","C","A","B","D","B"], n = 1

Output : 6

Explanation A possible execution:

A -> B -> C -> D -> A -> B

No idle interval is needed as cooldown = 1.

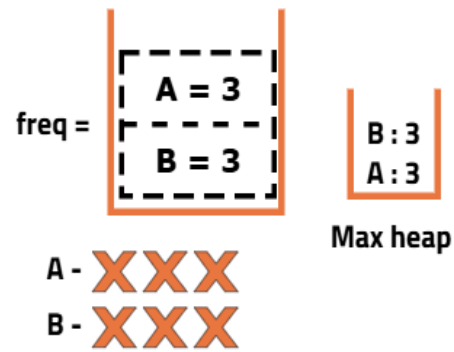
Approach

Algorithm

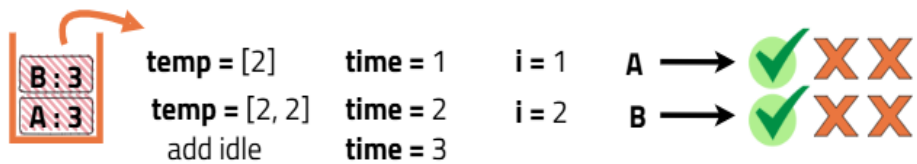
We want to execute all given tasks such that the same task type appears at least n time units apart. One brute force way to do this is by simulating the process. We use a max heap to always pick the task with the highest remaining frequency. In each unit of time, we execute a task if possible otherwise, we insert idle time. This approach ensures we respect the cooldown constraint but may add unnecessary idle time due to its greedy nature.

- Count the frequency of each task using a hash map.
- Push all frequencies into a max-heap (priority queue).
- While the heap is not empty:
 - Initialize an empty temporary list to hold tasks in current cycle.
 - For a total of $(n + 1)$ units of time, try to execute the most frequent remaining task:
 - Pop the task with highest frequency from heap.
 - Decrement its frequency and store it in temporary list if it still has counts left.
 - Increment total time for each executed or idle unit.
 - Push all tasks from the temporary list back into the heap.
- Return total time taken including idles.

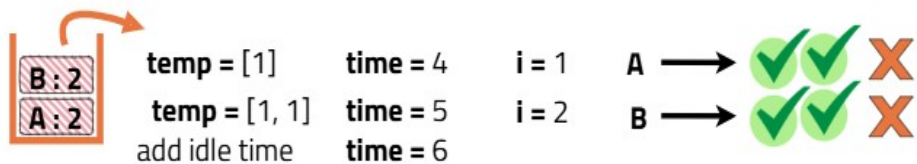
tasks = [A, A, A, B, B, B] n = 2



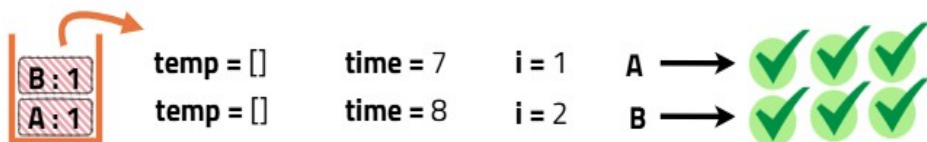
cycle = 3, i = 0, time = 0



cycle = 3, i = 0, time = 3



cycle = 3, i = 0, time = 6



heap empty, exit

time = 8

Code

```
import java.util.*;
class Solution {
    public int leastInterval(char[] tasks, int n) {

        // Step 1: Count frequency of each task
        Map<Character, Integer> freq = new HashMap<>();
        for (char task : tasks) {
            freq.put(task, freq.getDefault(task, 0) + 1);
        }

        // Step 2: Create a max heap (priority queue in reverse order)
        PriorityQueue<Integer> maxHeap = new
PriorityQueue<>(Collections.reverseOrder());
        for (int count : freq.values()) {
            maxHeap.add(count);
        }

        // Total time required
        int time = 0;

        // Step 3: Process tasks in cycles of size (n + 1)
        while (!maxHeap.isEmpty()) {

            // Temporary list to store tasks of current cycle
            List<Integer> temp = new ArrayList<>();

            // Set cycle size as cooldown + 1
            int cycle = n + 1;

            // Track number of tasks processed in current cycle
            int i = 0;

            // Run up to (n+1) tasks or until heap is empty
            while (i < cycle && !maxHeap.isEmpty()) {

                // Pick the most frequent task
                int count = maxHeap.poll();

                // Decrease frequency since task is used once
                count--;

                // If task still remains, store it for next cycle
                if (count > 0) {
                    temp.add(count);
                }

                // Count 1 unit time for this task
                time++;
                i++;
            }

            // Step 4: Push remaining tasks from temp back into the heap
            for (int rem : temp) {
                maxHeap.add(rem);
            }

            // Step 5: If heap is not empty, add idle time
            if (!maxHeap.isEmpty()) {
                time += (cycle - i);
            }
        }
    }
}
```

```

        // Return total time taken
        return time;
    }
}

// Driver class
class Main {
    public static void main(String[] args) {
        Solution sol = new Solution();

        // Input task list
        char[] tasks = {'A', 'A', 'A', 'B', 'B', 'B'};

        // Cooldown
        int n = 2;

        // Output the result
        System.out.println(sol.leastInterval(tasks, n));
    }
}

```

Complexity Analysis

Time Complexity: $O(N \log K)$, We count frequencies in $O(N)$ and use a max heap of K unique tasks. Each task can be pushed and popped from the heap multiple times resulting $(\log K)$ per operation.

Space Complexity: $O(K)$, We store task frequencies in a hashmap and use a max heap, both taking $O(K)$ space where K is the number of unique tasks.